CS2 Stats & Leaderboard: Setup Guide

For Trevor · adds Leetify-style stats, a leaderboard website and match scorecards to the existing match-bot

What this adds

Everything runs next to the CS2 server and uses the **same Discord bot** you already have. Nothing on the CS2 server or in AMP needs to change: MatchZy is already recording a demo of every map.

```
MatchZy records a demo -->  .../csgo/MatchZy/<date>_<matchid>_<map>_team_x_vs_team_y.dem
                                |  (read-only; checked every 20s, read once the file stops growing)
                                v
  stats container    - parses the demo, stores stats in SQLite (stats-data/stats.db)
                     - serves the leaderboard website on port 8080
                                |  (Docker's internal network)
                                v
  matchbot container - /leaderboard, /stats, /match, /matches, /link ...
                     - posts a scorecard to the stats channel when a match finishes
```

Only real, finished competitive maps count. Warmup, knife rounds, retakes, wingman, deathmatch, abandoned games and AFK players are filtered out automatically.

Before you start

- Docker with the compose plugin on the server (the bot already runs this way).
- About 400 MB of disk for the new image, plus a few MB for the database.
- A Discord channel for match scorecards (for example **#cs2-stats**).
- MatchZy demo recording left on. It's on by default (`matchzy_demo_recording_enabled true`), and the server log shows `[StartDemoRecoding]` lines.

Step 1: Get the new code

Sam will send these changes as a pull request on your `match-bot` repo. Merge it on GitHub, then on the server:

```
cd ~/match-bot          # wherever the bot lives
git pull
```

If you'd rather try it before merging, pull Sam's branch instead:

```
git remote add sam https://github.com/sd3mps/match-bot.git
git fetch sam
git checkout -b cs2-stats sam/cs2-stats
```

Step 2: Find MatchZy's demo folder

The stats container reads demos straight from the AMP instance. On a default AMP install the folder is:

```
/home/amp/.ampdata/instances/Counter-Strike201/counter-strike2/730/game/csgo/MatchZy
```

The instance name (`Counter-Strike201`) may differ. To find the folder for sure:

```
sudo find / -type d -name MatchZy -path "*csgo*" 2>/dev/null
```

```
ls -lh <that folder> | tail        # should list .dem files
```

Step 3: Add the new settings to .env

Open the bot's `.env` (`nano .env`). Keep everything already in it, and add these lines. `.env.example` lists them all with comments.

```
STATS_CHANNEL_ID=123456789012345678
STATS_API_KEY=<paste the output of: openssl rand -hex 24>
MATCHZY_DEMO_DIR=/home/amp/.ampdata/instances/Counter-Strike201/counter-strike2/730/game/csgo/MatchZy
STATS_PORT=8080
STATS_PUBLIC_URL=http://YOUR-PUBLIC-IP:8080
```

Setting	What it is
`STATS_CHANNEL_ID`	Where scorecards are posted. In Discord turn on Developer Mode (Settings › Advanced), then right-click the channel › Copy Channel ID.
`STATS_API_KEY`	A shared password between the bot and the stats service. Any long random string; both containers read it from this one file.
`MATCHZY_DEMO_DIR`	The folder from Step 2. Mounted read-only, so nothing can delete or change demos.
`STATS_PORT`	Port for the website on the host. Change it if 8080 is taken.
`STATS_PUBLIC_URL`	The address friends open in a browser (see Step 5). Used for the link buttons in Discord. Leave it empty until the site is reachable; the buttons are hidden while it's empty.
`STEAM_API_KEY`	Optional. Lets /link accept custom steamcommunity.com/id/name links. Get one at steamcommunity.com/dev/apikey.

Keep the .env file private. It holds the Discord token, the server password and the API key. It's already in .gitignore.

Step 4: Start it

```
docker compose up -d --build
docker compose logs -f stats       # Ctrl-C to stop watching
```

On the first start the stats service:

- creates `stats-data/stats.db`, seeded with the four games Sam backfilled from the server logs;
- imports every demo already in the MatchZy folder, quietly (no scorecard spam). Demo versions replace the log-based Wrecked and Ancient games;
- then checks for new demos every 20 seconds.

You should see lines like `serving on :8080` and `imported ..._de_ancient_....dem as match #5`. Test the website from the server itself:

```
curl http://localhost:8080/api/health      # {"ok": true, "matches": ...}
```

The bot registers its new slash commands on every start (same as before), so **/leaderboard**, **/stats** and the rest show up in Discord within a few seconds.

Step 5: Make the website reachable for friends

Pick one. After changing `STATS_PUBLIC_URL`, run `docker compose up -d` so the bot picks it up.

Option A: Port forward (simplest)

- On the router, forward TCP port **8080** to the server's LAN IP.
- If the server has a firewall: `sudo ufw allow 8080/tcp`.
- Set `STATS_PUBLIC_URL=http://<public IP>:8080`. Find the public IP with `curl ifconfig.me`.
- Note: people on the same home network as the server may need the LAN IP instead (the same NAT issue as the CS2 connect button).

Option B: Cloudflare Tunnel (HTTPS, no open ports)

Free, gives a proper https:// address, and doesn't expose your home IP. A named tunnel on your own domain stays up permanently. To try it quickly:

```
cloudflared tunnel --url http://localhost:8080     # prints a https://....trycloudflare.com address
```

Option C: An existing reverse proxy

If you already run Caddy or nginx for other services, point a subdomain at port 8080. For Caddy:

```
stats.yourdomain.com {
    reverse_proxy localhost:8080
}
```

> The website is read-only. Linking accounts, deleting matches and the scorecard queue all need STATS_API_KEY, which only the bot has.

Step 6: Discord permissions

The bot needs **View Channel**, **Send Messages** and **Embed Links** in the stats channel. No new privileged intents are needed. Then tell everyone to run **/link** once with their Steam profile URL. That connects their Discord account to their Steam ID, so **/stats** works with no arguments and the MVP gets tagged in each scorecard.

New Discord commands

Command	What it does
`/leaderboard [stat] [period] [min_matches]`	Ranked table, 10 players per page with Prev/Next buttons, plus a button to the full website. Rank by Rating 3.0, K/D, ADR, KAST, swing, headshots, clutches, utility, Leetify-style category scores and more.
`/stats [player] [member]`	A player's profile: rating, category percentiles, duels, utility and recent matches. Defaults to you once you've linked.
`/match [id]`	Full scorecard for the latest match, or a specific one.
`/matches`	The last 10 matches, with links to each scorecard.
`/link <steam> · /unlink`	Connect or disconnect your Discord and Steam accounts.
`/deletematch <id>`	Admins only (Manage Server). Removes a test or bad game from the stats. The demo file is untouched.

Command	What it does
`/rescan`	Admins only. Checks the demo folder immediately instead of waiting up to 20 seconds.

After every match, the bot posts the scorecard to the stats channel on its own, about a minute after the map ends: both teams' K-D-A, ADR and rating, plus highlights (MVP, entry king, best clutch, aces, nade merchant, flash god). If the MVP has run /link, they get tagged.

Disk space and resources

Thing	Size	Notes
Stats database	a few KB per match	About 10 MB after 1,000 matches. Only per-player totals and a round-by-round result are stored, not the demos.
Stats image	~400 MB	Python plus the demo parser. One-time.
CPU / RAM	capped at 1 core / 1 GB	Parsing one demo takes a few seconds. Idle otherwise.
Container logs	capped at 10 MB	Rotated automatically.
MatchZy demos	50–100 MB per map	Written by MatchZy, not by this. They pile up over time; see below.

The stats service never deletes demos (it can't: the folder is mounted read-only). Once a demo is imported, the stats live in the database, so old demos can be removed safely. To keep only the last 30 days, add this to the **amp** user's crontab (`sudo crontab -u amp -e`):

```
DEMOS=/home/amp/.ampdata/instances/Counter-Strike201/counter-strike2/730/game/csgo/MatchZy
0 5 * * * find "$DEMOS" -name '*.dem' -mtime +30 -delete
```

> Use the same folder as MATCHZY_DEMO_DIR. Keep demos you might want to rewatch somewhere else first.

Backups and updates

- **Backup:** everything that matters is `stats-data/stats.db`. Copy it somewhere now and then.
- **Update:** `git pull && docker compose up -d --build`. The database upgrades itself.
- **Start over:** stop with `docker compose down`, delete `stats-data/`, start again. It re-imports every demo still in the folder.

Troubleshooting

Problem	Check
compose says “Set MATCHZY_DEMO_DIR in .env”	The setting is missing from .env (Step 3).
Website won't load from outside	Try http://localhost:8080 on the server first. If that works, it's the port forward or firewall (Step 5).
No scorecard after a match	`docker compose logs stats`: look for “imported” or “skipped … (reason)”. Then `docker compose logs matchbot`: look for “[match feed]”. Check STATS_CHANNEL_ID, STATS_API_KEY and the channel permissions.
A real match was skipped	The log gives the reason (e.g. “only 9 rounds”, “team sizes 3v5”, “unfinished 12:10”). For MR8 short matches set WIN_ROUNDS=9. For smaller games lower MIN_TEAM_SIZE.
“The stats service isn't reachable” in Discord	`docker compose ps`: both containers should be running. Restart with `docker compose up -d`.
Permission denied reading demos	The container reads as root, which normally works. With rootless Docker, give read access: `sudo chmod -R o+rX <MatchZy folder>`.
Commands don't show up	`docker compose logs matchbot` should say “Commands registered!”. Discord can take a minute; restarting the Discord app helps.

How the numbers work

- **Rating 3.0:** an approximation of HLTV's Rating 3.0 (Swing 33%, Kills 25%, Survival 14%, KAST 12%, Damage 11%, Multi-kills 5%), eco-adjusted. **1.00 = the average player in our games**, not pros.
- **Round swing:** how much a player's kills and deaths changed their team's chance of winning each round.
- **Rating 2.0:** the older public HLTV formula, shown next to it for comparison.
- **Category scores** (Firepower, Entrying, Trading, Clutching, Utility): 0–100 percentiles within the group, like Leetify's profile.
- Kills and deaths were checked against the server's own scoreboard for every player on four maps, and matched.